KhaKhlong POS (ค้าคล่อง)

Progressive Web Application Point-of-Sale System for Restaurants

PROJECT DEFENSE CHEAT SHEET

สถาปัตยกรรมระบบ	ความปลอดภัย	ออฟไลน์ & ซิงก์	สถานะโปรเจกต์
React 18 + NestJS 11 + PostgreSQL 18	RLS Multi-tenancy + RBAC + PIN Lock	PWA + Dexie (IndexedDB) + Outbox	เสร็จสมบูรณ์ 100% (P0 / P1 / P2 ครบ)

หมวดที่ 1 ภาพรวมโปรเจกต์ ปัญหา และจุดเด่น (Problem Statement & Value Proposition)

Q1 ทำไมถึงทำโปรเจกต์นี้? POS ในท้องตลาดมีเยอะมาก ทำไมต้องทำเอง และมีจุดเด่นอย่างไร? STRATEGY

- **ลดต้นทุนอุปกรณ์ (Hardware Agnostic & Zero License Fee):** POS ทั่วไปในท้องตลาดมักผูกขาดกับฮาร์ดแวร์เฉพาะ (เช่น บังคับซื้อเครื่อง POS เฉพาะทางราคาแพง หรือผูกกับ iPad) แต่ *KhaKhlong* พัฒนาเป็น **PWA (Progressive Web App)** ใช้งานได้ผ่านเว็บเบราว์เซอร์บนทุกอุปกรณ์ ทั้งคอมพิวเตอร์ตั้งโต๊ะ, แท็บเล็ต, สมาร์ตโฟน และรองรับเครื่องพิมพ์ใบเสร็จความร้อนทั้ง 80mm และ 58mm
- **คุ้มครองความปลอดภัยและป้องกันการทุจริต (Anti-Fraud & Audit Compliance):** ปัญหาใหญ่ที่สุดของร้านอาหารคือการทุจริตจากการยกเลิกบิล (Void), ให้ส่วนลด/อาหารฟรี (Comp) และเงินในเก๊ะหาย ระบบเราบังคับใช้ **Two-Person PIN Rule** (ผู้จัดการ/เจ้าของร้านต้องใส่ PIN อนุมัติ), มีระบบล็อกรหัสผ่าน/PIN เมื่อกดผิด, และบันทึก **Audit Logs** ละเอียดทุกขั้นตอน
- **ทนทานต่อเน็ตหลุด (Offline-First Order Resilience):** รองรับการสั่งอาหารและเปิดโต๊ะได้ต่อเนื่องแม้สัญญาณเน็ตจะขาดหายชั่วคราว ข้อมูลไม่สูญหายและซิงก์ขึ้นเซิร์ฟเวอร์อัตโนมัติเมื่อออนไลน์

จุดที่เน้นตอบอาจารย์: เราไม่ได้สร้างแค่โปรแกรมคิดเงิน แต่สร้าง *"ระบบควบคุมการเงินและความปลอดภัยของร้านอาหารที่เข้าถึงง่าย ต้นทุนต่ำ แต่มีเสถียรภาพระดับ Enterprise"*

Q2 ผู้ใช้งานในระบบมีใครบ้าง และแบ่งสิทธิ์ (Roles & Permissions) อย่างไร? RBAC

ระบบใช้โมเดล **RBAC (Role-Based Access Control)** แบ่งออกเป็น 3 บทบาทหลัก:

- **Owner (เจ้าของร้าน):** สิทธิ์สูงสุด จัดการข้อมูลร้าน, จัดการผู้ใช้งานและพนักงาน, ดูรายงานยอดขายภาพรวมทั้งหมด, ตรวจรับรองกะการเงิน (Verify Shift), และเข้าถึงประวัติความปลอดภัย (Audit Logs)
- **Manager (ผู้จัดการร้าน):** บริหารจัดการเมนู, หมวดหมู่, โต๊ะอาหาร, อนุมัติรายการ Void/Comp ด้วย PIN, และตรวจสอบกะการเงินของพนักงาน
- **Employee (พนักงาน/แคชเชียร์):** สิทธิ์จำกัดเฉพาะหน้างาน (9 สิทธิ์) ได้แก่ เปิดบิล, รับออเดอร์ส่งครัว, ชำระเงิน, ดูเฉพาะกะของตนเอง (report:view_own_shift) แต่ไม่มีสิทธิ์ยกเลิกบิลหรือดูข้อมูลความปลอดภัย

การบังคับใช้สิทธิ์: คุ้มครองทั้งฝั่ง Backend ด้วย @RequirePermissions / @RequireAnyPermission และฝั่ง Frontend ด้วย RouteGuard และ Dynamic UI Elements

หมวดที่ 2 สถาปัตยกรรมระบบและเทคโนโลยี (Architecture & Multi-Tenancy)

Q3 ทำไมถึงเลือกใช้ Tech Stack ชุดนี้? (React, NestJS, PostgreSQL 18, Dexie) TECH STACK

- **Frontend (React 18 + Vite + Tailwind v4):** ได้ความเร็วสูงในการคอมไพล์ (Vite HMR), Type Safety เต็มรูปแบบด้วย TypeScript, Tailwind v4 ให้ UI ที่ตอบสนองลื่นไหล โทนคาเฟ่อบอุ่น (Warm Coffee) และออกแบบสอดคล้องกับหลัก Human Interface Guidelines สำหรับงานสัมผัส (Touch POS)
- **Backend (NestJS 11):** โครงสร้างแบบ Enterprise Modular Architecture แบ่งแยก Controller → Service → Repository ชัดเจน มี Dependency Injection (DI) และ DTO Validation ช่วยให้ทดสอบง่ายและขยายระบบได้ยั่งยืน
- **Database (PostgreSQL 18):** มีความสมบูรณ์ด้านธุรกรรมแบบ ACID สูง รองรับ Row-Level Security (RLS) ที่แข็งแกร่ง และมี DB Trigger/View ช่วยคำนวณภาษีและยอดเงินได้อย่างแม่นยำ
- **Client Database (Dexie.js / IndexedDB):** เป็น Local Database ในเบราว์เซอร์ สำหรับแคช Snapshot เมนู ผังโต๊ะ และจัดการคิวคำสั่งสั่งอาหารออฟไลน์ (Outbox Queue)

Q4 ระบบจัดการแยกข้อมูลร้านค้า (Multi-tenancy) อย่างไร? ป้องกันข้อมูลรั่วไหลได้อย่างไร? RLS HIGHLIGHT

- เราเลือกใช้สถาปัตยกรรม **Shared Database, Shared Schema with Row-Level Security (RLS)** ของ PostgreSQL
- ทุกตารางที่มีข้อมูลของร้านจะมีคอลัมน์ tenant_id กำกับ และเปิดใช้ RLS ผ่านฟังก์ชัน app_tenant_id()
- **กระบวนการทำงาน:** เมื่อ Client ส่งคำขอพร้อม JWT → Backend จะถอดรหัส Token และเซ็ตค่า Session Variable (SET LOCAL app.current_tenant_id = ...) ใน PostgreSQL Connection นั้น
- PostgreSQL จะบังคับกรองข้อมูลให้เห็นเฉพาะร้านนั้นโดยอัตโนมัติในระดับ Database Engine **แม้ผู้พัฒนาจะเผลอเขียนคำสั่ง SELECT/UPDATE โดยไม่ได้ใส่ WHERE tenant_id = ... ข้อมูลก็ไม่มีวันรั่วไหลข้ามร้านเด็ดขาด (Zero Data Leakage)**

หมวดที่ 3 ความปลอดภัยและการควบคุมธุรกรรม (Security, Auth & Concurrency)

Q5 ระบบป้องกันการทุจริต Void / Comp และบิลลูกค้าหนี (Walkout) ทำงานอย่างไร?

FRAUD PREVENTION

- **Two-Person PIN Rule:** พนักงานแคชเชียร์เป็นผู้กดเลือกรายการที่จะ Void (ยกเลิก) หรือ Comp (ให้ฟรี) แต่ระบบจะแสดง Modal บังคับให้ป้อน **PIN 6 หลักของผู้จัดการหรือเจ้าของร้าน** พร้อมระบุเหตุผล
- **Approver Invariant:** ในฐานข้อมูลจะบันทึก voided_by หรือ comped_by เป็น User ID ของ **"ผู้อนุมัติ"** เพื่อให้สามารถตรวจสอบย้อนหลังได้ชัดเจนว่าใครเป็นคนอนุมัติ
- **Close Unpaid (Walkout):** หากลูกค้าหนีโต๊ะ มีเส้นทางปิดบิลพิเศษที่ต้องใช้ PIN ผู้อนุมัติและบันทึกหมายเหตุ เพื่อให้ระบบปิดบิลและคืนสถานะโต๊ะกลับเป็นว่าง (Available) ได้อย่างถูกต้องโดยไม่ทำให้โต๊ะค้างสถานะ Occupied ตลอดไป

Q6 ทำไมต้อง durableQuery ในการนับจำนวนครั้งที่กด PIN ผิด? (ไฮไลท์เชิงเทคนิค!)

ARCHITECTURE HIGHLIGHT

- **ปัญหาเดิม:** เมื่อพนักงานใส่ PIN อนุมัติผิด Backend จะส่ง 403 Forbidden ทำให้ Database ทำการ **Rollback Transaction** ทั้งหมด ส่งผลให้ตัวนับจำนวนครั้งที่ใส่รหัสผิด (Failed Attempts) ถูก Rollback หายไปด้วย ทำให้ผู้ร้ายสามารถสุ่มเดา PIN (Brute-force) ได้ไม่จำกัดครั้ง!
- **วิธีแก้เชิงวิศวกรรม:** เราสร้าง DbContextService.durableQuery() ที่ใช้ Database Connection พิเศษทำงานนอก Transaction ของคำขอนั้น ทำหน้าที่เขียนตัวนับ pin_failed_attempts ลงตาราง users อย่างคงทนถาวร
- เมื่อใส่ PIN ผิดครบ 5 ครั้ง ระบบจะตั้งเวลาล็อกบัญชี 5 นาที (ส่งสถานะ 429 Too Many Requests) อย่างแท้จริง ข้าม Session และข้ามการ Restart เซิร์ฟเวอร์

Q7 Authentication ใช้ JWT แล้วจัดการเรื่อง Logout หรือเปลี่ยนรหัสผ่านอย่างไร? (Token Revocation)

AUTH PATTERN

- ปกติ JWT เป็น Stateless ทำให้ไม่สามารถยกเลิก Token กลางคันก่อนหมดอายุได้
- เราแก้ปัญหานี้ด้วย **token_version Pattern:** ในฐานข้อมูลตาราง users จะมีคอลัมน์ token_version (ตัวเลขจำนวนเต็ม) และใส่ค่านี้ไว้ใน Payload ของ JWT
- เมื่อผู้ใช้เปลี่ยนรหัสผ่าน, รีเซ็ตรหัสผ่าน หรือถูก Deactivate บัญชี ระบบจะเพิ่มค่า token_version = token_version + 1 ในฐานข้อมูลทันที
- เมื่อ Token เก่าส่งเข้ามาที่ jwtAuth.guard.ts จะถูกตรวจสอบกับค่าในฐานข้อมูล หากเวอร์ชันไม่ตรงกันจะถูกปฏิเสธทันที (Session เก่าจะหลุดหมดทั้งระบบ)

หมวดที่ 4 การเงิน การคำนวณ และการปิดกะ (Financial Invariants & Shifts)

Q8 "ทำไมถึงห้ามคำนวณ VAT, Service Charge และยอดรวมใน NestJS หรือ Client?" CORE INVARIANT

- **Single Source of Truth:** ป้องกันปัญหาการปัดเศษทศนิยม (Rounding Differences) ที่ไม่ตรงกันระหว่าง JavaScript (IEEE 754 Floating Point) และ Database
- **Tamper-Proof Security:** ป้องกันไม่ให้ฝั่ง Client หรือ API Caller แอบแก้ไขยอดเงิน ยอดภาษี หรือส่วนลดผ่าน Network Payload
- **Database Engine Responsibility:** ฐานข้อมูล PostgreSQL ผ่าน View v_order_item_totals และ Database Trigger เป็นผู้คำนวณยอดรวม, ส่วนลด, Service Charge, VAT 7%, และยอดปัดเศษสตางค์ตามกฎหมาย พร้อมทำ Snapshot บันทึกอัตราภาษีลงใบเสร็จ ณ วินาทีที่ชำระเงิน

Q9 กระบวนการชำระเงิน (Payment Workflow) ทำงานอย่างไร? ทำไมต้องมี Payment Attempt? 2-PHASE PATTERN

เราใช้สถาปัตยกรรมแบบ **2-Phase Payment Attempt** เพื่อความสมบูรณ์ทางการเงิน:

- **ขั้นที่ 1 (จองบิล):** เมื่อกดปุ่มคิดเงิน Frontend จะเรียก POST /payments/attempts เพื่อให้เซิร์ฟเวอร์คำนวณยอดเงินสุทธิ และทำการ **Lock สถานะบิล** ป้องกันไม่ให้มีการสั่งอาหารเพิ่ม หรือกดชำระเงินซ้ำซ้อนจากเครื่องอื่นพร้อมกัน (Double Payment Prevention)
- **ขั้นที่ 2 (รับเงิน/ปิดบิล):** แคชเชียร์เลือกวิธีชำระ (เงินสด หรือ QR PromptPay) แล้วยืนยันผ่าน POST .../attempts/:id/pay เมื่อชำระสำเร็จ Trigger จะปิดบิล, ออกเลขที่ใบเสร็จทางการ และคืนสถานะโต๊ะเป็นว่าง (Available) อัตโนมัติ
- **กรณียกเลิก:** หากลูกค้าเปลี่ยนใจหรือปิด Modal ชำระเงิน Frontend จะเรียก POST .../attempts/:id/cancel เสมอเพื่อปลดล็อกบิลกลับสู่สภาพเดิม

Q10 ระบบกะการเงิน (Shift Management) ควบคุมเงินสดในร้านอย่างไร? CASH CONTROL

- **เปิดกะ (Open Shift):** บังคับป้อนเงินทอนตั้งต้น (Opening Cash)
- **สมุดเงินสด (Cashbook):** ทุกการขายเงินสด, เงินเข้า (Cash In) และเบิกเงินออก (Cash Out) จะถูกบันทึกความเคลื่อนไหวลงตาราง cash_movements แบบ Immutable
- **ปิดกะ (Close Shift & Blind Count):** แคชเชียร์ต้องนับเงินสดจริงในเก๊ะมาป้อนโดยไม่แสดงยอดที่ระบบคาดหวังให้เห็นล่วงหน้า เพื่อป้องกันการตกแต่งตัวเลข
  - ถ้ายอดเงินตรง: กะจะปิดสมบูรณ์ทันที (Status: closed)
  - ถ้ายอดขาดหรือเกิน: กะจะติดสถานะ pending_verification ต้องให้ Manager หรือ Owner ตรวจสอบและใส่ PIN ยืนยันรับทราบผลต่าง
- **Shift Escalation Cron:** หากกะติดสถานะรอตรวจสอบเกินกว่าเวลาที่ร้านตั้งไว้ (Default 4 ชม.) Background Job จะส่งต่อกะให้ Owner ตรวจสอบโดยตรงอัตโนมัติ

หมวดที่ 5 การทำงานออฟไลน์และ PWA (Offline-First & PWA)

Q11 PWA และ Offline Mode ทำงานอย่างไร? ข้อมูลหายไหมถ้าเน็ตหลุดกลางคัน? DEXIE & OUTBOX

- **Service Worker App Shell:** ใช้ Workbox แคชไฟล์หน้าเว็บทั้งหมด ทำให้เปิดแอพได้ทันทีแม้ไม่มีอินเทอร์เน็ต
- **Local Snapshots (Dexie.js / IndexedDB):** ข้อมูลเมนูอาหาร, ผังโต๊ะ และกะปัจจุบัน จะถูกแคชลง IndexedDB ในเครื่อง ด้วยกลยุทธ์ *Network-First Fallback* (ยิงเซิร์ฟเวอร์ก่อน หากเน็ตหลุดจะดึง snapshot ล่าสุดมาแสดงแทน)
- **Outbox Queue Pattern:** หากพนักงานเปิดโต๊ะหรือสั่งอาหารขณะออฟไลน์ คำสั่งจะถูกบันทึกลง Dexie Outbox ทันที พร้อมทำ Optimistic Update ผังโต๊ะในเครื่อง เพื่อให้พนักงานสั่งอาหารเพิ่มที่โต๊ะเดิมต่อได้
- **Atomic Synchronization (Web Locks):** เมื่อเน็ตกลับมาออนไลน์ OutboxRunner จะวนลูปส่งคำสั่งขึ้นเซิร์ฟเวอร์ตามลำดับ (FIFO) โดยใช้ **Web Locks API** ป้องกันไม่ให้หลายแท็บเบราว์เซอร์แย่งกันส่งข้อมูลชนกัน
- **Data Guard:** ระบบมี Guard บล็อกการ Logout, ปิดกะ หรืออัปเดต Service Worker หากยังมีคำสั่งค้างอยู่ใน Outbox เพื่อป้องกันข้อมูลสูญหาย 100%

Q12 ทำไมถึงไม่อนุญาตให้คิดเงิน/ชำระเงิน (Payment) ขณะออฟไลน์? FINANCIAL DECISION

เป็นการตัดสินใจเชิงธุรกิจและความปลอดภัยทางการเงิน (Financial Integrity):

- **ความถูกต้องของเงินจริง:** การชำระเงินจำเป็นต้องยืนยันยอดเงินจริง (เช่น เงินสดเข้าเก๊ะ หรือ QR PromptPay ที่ลูกค้าโอนจริง)
- **Sequential Receipt Numbering:** เลขที่ใบเสร็จทางการของร้านอาหารตามกฎหมายต้องรันต่อเนื่องห้ามซ้ำและห้ามกระโดด ซึ่งต้องให้ Database ศูนย์กลางเป็นผู้ออกเลข
- **ป้องกันพนักงานทุจริต:** หากเปิดให้ออฟไลน์แคชเชียร์อาจรับเงินสดจากลูกค้าแล้วล้างแคชทิ้งโดยไม่ซิงก์ข้อมูล
- **สรุป:** ระบบจึงอนุญาตให้ออฟไลน์เฉพาะการ *"รับออเดอร์และส่งครัว"* ส่วนการคิดเงินต้องทำแบบ Online เท่านั้น

หมวดที่ 6 การทดสอบและการประกันคุณภาพ (Testing & QA)

Q13 คุณทดสอบโปรเจกต์นี้อย่างไรเพื่อยืนยันว่าระบบทำงานถูกต้องและพร้อมใช้งานจริง? QUALITY ASSURANCE

- **Compile & Type Safety:** ผ่านการตรวจสอบ Type 100% ทั้งฝั่ง Backend (tsc --noEmit) และ Frontend (tsc -b)
- **Unit Tests (Jest):** ทดสอบฟังก์ชันและเซอร์วิสที่มีความซับซ้อน เช่น การเปลี่ยนสถานะกะ, การตรวจสอบ PIN, และ Shift Escalation Service
- **End-to-End Tests (Node.js .mjs):** เราพัฒนาชุดทดสอบ E2E เขียนเป็นสคริปต์ Node.js รันยิงคำขอกับ NestJS และ PostgreSQL จริงมากกว่า 13 ชุดทดสอบ ครอบคลุม Business Critical Flows เช่น:
  - e2e-void-comp.mjs: ทดสอบสิทธิ์ PIN อนุมัติ Void/Comp และการถูกล็อกบัญชี 31 ข้อผ่านครบ 100%
  - e2e-cancel-order.mjs: ทดสอบการยกเลิกบิลและลูกค้าหนี (Walkout)
  - e2e-merge-tables-payment.mjs: ทดสอบการรวมหลายโต๊ะคิดเงินพร้อมกัน
  - e2e-token-version.mjs: ทดสอบการตัดเซสชันเมื่อเปลี่ยนรหัสผ่าน
  - e2e-audit-logs.mjs, e2e-receipt.mjs, e2e-printer-config.mjs, e2e-category-reorder.mjs

โบนัส ขั้นตอนการสาธิตหน้าห้องสอบที่แนะนำ (5-Step Demo Script)

ขั้นตอน	หน้าจอ / การกระทำ	สิ่งที่ต้องอธิบายให้อาจารย์ฟัง	ไฮไลท์เทคโนโลยี
Step 1	**Store Finder & Login** เข้าผ่าน URL ร้าน และเปิดกะ	แสดงระบบ Multi-tenant ผ่าน URL slug และการเปิดกะระบุเงินทอนตั้งต้น (Opening Cash)	Tenant Isolation + Shift Management
Step 2	**ผังโต๊ะ & สั่งอาหาร** เลือกโต๊ะ สั่งเมนู + ตัวเลือก (Modifier)	แสดง UI ตอบสนองรวดเร็ว การเลือกเมนู ตัวเลือกเสริม และการส่งครัว	Optimistic UI + KDS Sync
Step 3	**Void / Comp อนุมัติ** ขอยกเลิกหรือให้ฟรี	แสดง Modal ขอ PIN ผู้จัดการ อธิบายเรื่องการบันทึกสิทธิ์ผู้อนุมัติ และระบบป้องกัน PIN ผิดด้วย durableQuery	Two-Person Rule + Fraud Prevention
Step 4	**คิดเงิน & พิมพ์ใบเสร็จ** ชำระเงินสด / QR Code	แสดง 2-Phase Payment Attempt ยอดเงินถูกคำนวณจาก DB Trigger โต๊ะคืนสถานะว่างอัตโนมัติ และแสดงใบเสร็จ 80mm/58mm	Financial Integrity + Thermal Print
Step 5	**สลับเป็น Owner ดู Audit** ประวัติความปลอดภัย & รายงาน	แสดง Audit Logs ที่บันทึกการกระทำเมื่อครู่ และหน้ารายงานยอดขายตามกะ/ช่วงเวลา พร้อมส่งออก CSV ภาษาไทยไม่เพี้ยน	Audit Compliance + CSV Export UTF-8 BOM

จัดทำขึ้นสำหรับโปรเจกต์ KhaKhlong (ค้าคล่อง) — ขอให้การสอบจบโปรเจกต์ผ่านพ้นไปด้วยคะแนนยอดเยี่ยม!